

Build: Standard Output Contract

Unified JSON Response for Frontend, AI Narratives, and PDF Reports

Objective

Create one consistent **EnterprateAI Intelligence Output Contract** that every major intelligence workflow can use.

This contract must support:

- frontend dashboard
- engine results
- risk display
- recommendations
- scenario comparison
- Adaptive Scenario Intelligence
- AI narratives
- PDF reports
- database persistence

This becomes the **single intelligence language** of EnterprateAI.

Recommended Folder

`/src/lib/contracts`

Suggested structure:

```
/src/lib/contracts
├─ intelligence-output.types.ts
├─ intelligence-output.schema.ts
```

```
|— intelligence-output.examples.ts  
|— index.ts
```

1. Why This Matters

Without a standard output contract:

- frontend becomes inconsistent
- PDF reports become difficult
- AI narratives hallucinate or lack structure
- simulation outputs become hard to compare
- engine outputs cannot be aggregated cleanly

With this contract:

Engines → Orchestrator → Standard Output → Frontend / AI / PDF / Database

2. Standard Output Shape

```
export interface EnterpriseIntelligenceOutput {  
  meta: IntelligenceMeta;  
  
  business: BusinessContext;  
  
  executiveSummary: ExecutiveSummary;  
  
  scores: ScoreBlock;  
  
  classifications: ClassificationBlock;  
  
  engineResults: EngineResultsBlock;  
  
  fragility: FragilityBlock;
```

```
risks: RiskBlock;

recommendations: RecommendationBlock;

scenarios?: ScenarioBlock;

adaptiveIntelligence?: AdaptiveIntelligenceBlock;

narratives?: NarrativeBlock;

reportData?: ReportDataBlock;

confidence: ConfidenceBlock;

trace: TraceBlock;
}
```

3. Meta Block

```
export interface IntelligenceMeta {
  outputId: string;
  businessId: string;
  generatedAt: string;
  outputType:
    | "assessment"
    | "simulation"
    | "adaptive_insight"
    | "report"
    | "dashboard";
  rulesetVersion: string;
  engineVersion: string;
  source:
    | "manual"
    | "integration"
    | "simulation"
    | "asi"
```

```
    | "system";  
}
```

4. Business Context Block

```
export interface BusinessContext {  
  businessName?: string;  
  sector?: string;  
  businessStage?: string;  
  countryCode?: string;  
  currency?: string;  
}
```

Purpose:

Used by:

- frontend headers
- AI narrative context
- PDF report cover/summary

5. Executive Summary Block

```
export interface ExecutiveSummary {  
  headline: string;  
  summary: string;  
  currentBusinessState: string;  
  keyStrength: string;  
  keyWeakness: string;  
  strategicPriority: string;  
  nextBestActions: string[];  
}
```

Example:

```
{
  "headline": "The business is promising but exposed.",
  "summary": "The business has strong growth potential but weak cash
resilience.",
  "currentBusinessState": "Scaling with fragility",
  "keyStrength": "Growth potential",
  "keyWeakness": "Cash runway",
  "strategicPriority": "Protect cash runway before scaling.",
  "nextBestActions": [
    "Reduce fixed costs",
    "Run a pricing scenario",
    "Delay hiring until runway improves"
  ]
}
```

6. Score Block

```
export interface ScoreBlock {
  viabilityScore: number;
  survivalScore: number;
  stabilityScore: number;
  growthScore: number;
  businessIntelligenceScore: number;
  fragilityIndex: number;

  scoreTrend?: {
    direction: "improving" | "stable" | "declining" | "volatile";
    previousScore?: number;
    change?: number;
  };
}
```

7. Classification Block

```
export interface ClassificationBlock {
  primary:
    | "HIGH_PERFORMING"
    | "STRONG_BUT_NEEDS_OPTIMISATION"
    | "AT_RISK"
    | "CRITICAL";

  structural:
    | "BALANCED_BUSINESS"
    | "VIAABLE_BUT_CASH_CRITICAL"
    | "GROWTH_READY_BUT_UNSTABLE"
    | "STABLE_BUT_STAGNANT"
    | "PROFITABLE_BUT_FRAGILE"
    | "SCALING_WITH_FRAGILITY"
    | "WEAK_FOUNDATION"
    | "CRITICAL_BUSINESS_STATE";

  fragility:
    | "RESILIENT"
    | "STABLE_WITH_EXPOSURE"
    | "FRAGILE"
    | "HIGHLY_FRAGILE"
    | "CRITICAL_FRAGILITY";

  label: string;
}
```

8. Engine Results Block

```
export interface EngineResultsBlock {
  viability: StandardEngineResult;
  survival: StandardEngineResult;
  stability: StandardEngineResult;
}
```

```
    growth: StandardEngineResult;  
}
```

Standard Engine Result

```
export interface StandardEngineResult {  
    engine: "viability" | "survival" | "stability" | "growth";  
    score: number;  
    classification: string;  
    headline: string;  
    summary: string;  
    keyMetrics: Array<{  
        name: string;  
        value: number | string | null;  
        unit?: string;  
        explanation?: string;  
    }>;  
    riskFlags: StandardRiskFlag[];  
    recommendations: StandardRecommendation[];  
    confidence: StandardConfidence;  
}
```

9. Fragility Block

```
export interface FragilityBlock {  
    index: number;  
    classification: string;  
    dimensions: {  
        cashFragility: number;  
        revenueFragility: number;  
        customerFragility: number;  
        operationalFragility: number;  
        capacityFragility: number;  
        marketFragility: number;  
    };  
    topFragilityFactors: Array<{
```

```
        factorCode: string;
        dimension: string;
        severity: string;
        explanation: string;
        mitigationAction: string;
    }>;
    propagationRisks: Array<{
        source: string;
        path: string[];
        propagationScore: number;
        explanation: string;
    }>;
}
```

10. Risk Block

```
export interface RiskBlock {
    totalRisks: number;
    criticalRisks: number;
    highRisks: number;
    topRisks: StandardRiskFlag[];
    groupedRisks: Array<{
        category:
            | "commercial"
            | "financial"
            | "operational"
            | "customer"
            | "market"
            | "capacity"
            | "structural";
        risks: StandardRiskFlag[];
    }>;
}
```


Standard Risk Flag

```
export interface StandardRiskFlag {
  code: string;
  consolidatedCode?: string;
  severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL" | "VERY_HIGH";
  sourceEngines: string[];
  explanation: string;
  recommendedAction: string;
  priorityScore: number;
  confidence?: "LOW" | "MEDIUM" | "HIGH";
}
```

11. Recommendation Block

```
export interface RecommendationBlock {
  totalRecommendations: number;
  topRecommendations: StandardRecommendation[];
  groupedRecommendations: Array<{
    category:
      | "survival"
      | "profitability"
      | "growth"
      | "stability"
      | "risk_reduction"
      | "data_quality";
    recommendations: StandardRecommendation[];
  }>;
}
```

Standard Recommendation

```
export interface StandardRecommendation {
  id?: string;
  type: string;
}
```

```
category:
  | "survival"
  | "profitability"
  | "growth"
  | "stability"
  | "risk_reduction"
  | "data_quality";

priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
text: string;
reason: string;
expectedImpact?: string;
expectedImpactScore?: number;
implementationDifficulty?: "LOW" | "MEDIUM" | "HIGH";
suggestedScenario?: string;
sourceEngines: string[];
}
```

12. Scenario Block

```
export interface ScenarioBlock {
  scenarioId?: string;
  scenarioName?: string;
  scenarioType?: string;

  baseScores: {
    viability: number;
    survival: number;
    stability: number;
    growth: number;
    businessIntelligence: number;
    fragilityIndex: number;
  };

  simulatedScores: {
    viability: number;
    survival: number;
```

```
    stability: number;
    growth: number;
    businessIntelligence: number;
    fragilityIndex: number;
};

deltas: {
    viabilityDelta: number;
    survivalDelta: number;
    stabilityDelta: number;
    growthDelta: number;
    businessIntelligenceDelta: number;
    fragilityDelta: number;
};

riskChanges: Array<{
    code: string;
    change: "added" | "removed" | "updated";
    previousSeverity?: string;
    newSeverity?: string;
}>;

recommendationChanges: Array<{
    type: string;
    change: "added" | "removed" | "updated";
    previousPriority?: string;
    newPriority?: string;
}>;

decisionPath?: {
    rankingScore: number;
    reason: string;
    recommendedActions: string[];
};
}
```

13. Adaptive Intelligence Block

```
export interface AdaptiveIntelligenceBlock {
  triggers: Array<{
    triggerCode: string;
    triggerType: "risk" | "opportunity" | "trend";
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    reason: string;
  }>;

  generatedScenarios?: Array<{
    scenarioName: string;
    objective:
      | "improve_survival"
      | "reduce_fragility"
      | "increase_growth"
      | "improve_margin";
    expectedImpact: string;
  }>;

  bestDecisionPath?: {
    name: string;
    reason: string;
    expectedOutcome: string;
    recommendedActions: string[];
  };
}
```

14. Narrative Block

```
export interface NarrativeBlock {
  executiveNarrative?: string;
  riskNarrative?: string;
  recommendationNarrative?: string;
  scenarioNarrative?: string;
}
```

```
    confidenceNarrative?: string;  
}
```

Important:

Narratives are generated later by AI, but must be grounded only on this output object.

15. Report Data Block

```
export interface ReportDataBlock {  
  reportTitle?: string;  
  reportType?:  
    | "business_health_report"  
    | "simulation_report"  
    | "fragility_report"  
    | "adaptive_decision_report";  
  
  sections: Array<{  
    sectionKey: string;  
    title: string;  
    summary: string;  
    dataSource:  
      | "scores"  
      | "risks"  
      | "recommendations"  
      | "fragility"  
      | "scenario"  
      | "narrative";  
  }>;  
}
```

16. Confidence Block

```
export interface ConfidenceBlock {  
  overallLevel: "LOW" | "MEDIUM" | "HIGH";  
}
```

```
overallScore: number;
engineConfidence: {
  viability: "LOW" | "MEDIUM" | "HIGH";
  survival: "LOW" | "MEDIUM" | "HIGH";
  stability: "LOW" | "MEDIUM" | "HIGH";
  growth: "LOW" | "MEDIUM" | "HIGH";
  fragility: "LOW" | "MEDIUM" | "HIGH";
};
dataQualityNotes: string[];
}
```

Standard Confidence

```
export interface StandardConfidence {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
  penaltyApplied?: boolean;
}
```

17. Trace Block

```
export interface TraceBlock {
  orchestrationSequence: string[];
  engineVersions: Record<string, string>;
  rulesetVersion: string;
  inputSnapshotId?: string;
  assessmentId?: string;
  simulationRunId?: string;
  generatedBy: "orchestrator" | "scenario_engine" | "asi";
}
```

18. Example Final JSON Output

```
{
  "meta": {
    "outputId": "out_001",
    "businessId": "bus_001",
    "generatedAt": "2026-05-31T12:00:00Z",
    "outputType": "assessment",
    "rulesetVersion": "v1.0",
    "engineVersion": "v1.0",
    "source": "manual"
  },
  "business": {
    "businessName": "Alpha Advisory",
    "sector": "consulting",
    "businessStage": "early_revenue",
    "countryCode": "GB",
    "currency": "GBP"
  },
  "executiveSummary": {
    "headline": "The business is promising but exposed.",
    "summary": "The business has strong growth potential but weak cash resilience.",
    "currentBusinessState": "Scaling with fragility",
    "keyStrength": "Growth potential",
    "keyWeakness": "Cash runway",
    "strategicPriority": "Protect cash runway before scaling.",
    "nextBestActions": [
      "Reduce fixed costs",
      "Run a pricing scenario",
      "Delay hiring until runway improves"
    ]
  },
  "scores": {
    "viabilityScore": 78,
    "survivalScore": 42,
    "stabilityScore": 55,
    "growthScore": 81,
  }
}
```

```
    "businessIntelligenceScore": 65.35,
    "fragilityIndex": 68
  },
  "classifications": {
    "primary": "STRONG_BUT_NEEDS_OPTIMISATION",
    "structural": "SCALING_WITH_FRAGILITY",
    "fragility": "HIGHLY_FRAGILE",
    "label": "Strong but Scaling with Fragility"
  },
  "risks": {
    "totalRisks": 3,
    "criticalRisks": 0,
    "highRisks": 2,
    "topRisks": [
      {
        "code": "LOW_RUNWAY_RISK",
        "consolidatedCode": "FINANCIAL_SURVIVAL_RISK",
        "severity": "HIGH",
        "sourceEngines": ["survival"],
        "explanation": "The business has limited cash runway.",
        "recommendedAction": "Reduce burn and improve short-term cash
inflows.",
        "priorityScore": 88,
        "confidence": "MEDIUM"
      }
    ],
    "groupedRisks": []
  },
  "recommendations": {
    "totalRecommendations": 3,
    "topRecommendations": [
      {
        "type": "cash_preservation",
        "category": "survival",
        "priority": "HIGH",
        "text": "Improve cash runway before scaling aggressively.",
        "reason": "Growth potential is strong, but survival weakness
creates scaling risk.",
        "expectedImpact": "Improves survival resilience",
```



```
        "sourceEngines": ["survival", "growth"]
    },
],
"groupedRecommendations": []
},
"confidence": {
    "overallLevel": "MEDIUM",
    "overallScore": 66,
    "engineConfidence": {
        "viability": "MEDIUM",
        "survival": "LOW",
        "stability": "MEDIUM",
        "growth": "MEDIUM",
        "fragility": "MEDIUM"
    },
    "dataQualityNotes": [
        "Survival confidence is reduced due to limited cashflow
history."
    ]
},
"trace": {
    "orchestrationSequence": [
        "viability",
        "survival",
        "stability",
        "growth",
        "master",
        "fragility"
    ],
    "engineVersions": {
        "viability": "1.0.0",
        "survival": "1.0.0",
        "stability": "1.0.0",
        "growth": "1.0.0"
    },
    "rulesetVersion": "v1.0",
    "generatedBy": "orchestrator"
}
}
```

19. Acceptance Criteria

Codex should complete this when:

- all TypeScript interfaces are created
- Zod schema validation is added if Zod exists in project
- example outputs are provided
- frontend can safely consume the contract
- AI narrative layer can use this as grounding
- PDF report generator can use this as source
- no engine-specific output leaks directly to frontend without standardisation

20. Codex Prompt

Create the EnterprateAI Standard Intelligence Output Contract in `/src/lib/contracts`.

Create:

- `intelligence-output.types.ts`
- `intelligence-output.schema.ts`
- `intelligence-output.examples.ts`
- `index.ts`

Define TypeScript interfaces for:

- `EnterprateIntelligenceOutput`
- `IntelligenceMeta`
- `BusinessContext`
- `ExecutiveSummary`
- `ScoreBlock`
- `ClassificationBlock`
- `EngineResultsBlock`
- `StandardEngineResult`
- `FragilityBlock`
- `RiskBlock`

- StandardRiskFlag
- RecommendationBlock
- StandardRecommendation
- ScenarioBlock
- AdaptiveIntelligenceBlock
- NarrativeBlock
- ReportDataBlock
- ConfidenceBlock
- StandardConfidence
- TraceBlock

If Zod exists in the project, create matching Zod schemas.

Add at least one complete example output for a standard assessment.

Export all types and schemas from index.ts.

This contract will be used by:

- frontend
- AI narrative layer
- PDF reports
- API responses
- persistence mapping.